Part 6 - Pygame Car Crash Tutorial

Contents

Part 6 - Pygame Car Crash Tutorial.....1
CarCrash Class 2
Sound 5
Enemy Class.....7
What's Next? 8
Assignment Submission.....8

Time required: 30 minutes

Let's finish up our car game with backgrounds, sounds, fonts, and a scoring system.

CarCrash Class

```python
class CarCrash:
    # Add a new User event to increase speed
    # + 1 ensures that the USEREVENT is unique
    INC_SPEED = pygame.USEREVENT + 1

    # Constant for how much the speed increases each userevent
    SPEED_INCREASE = .01

    RED = (255, 0, 0)
    BLACK = (0, 0, 0)

    def __init__(self):
        # Initialize the pygame library
        pygame.init()

        # Set timer for INC_SPEED to increase speed every 2 seconds
        pygame.time.set_timer(CarCrash.INC_SPEED, 2000)

        # Setup a computer clock object
        self.CLOCK = pygame.time.Clock()

        # Create the game window
        self.surface = pygame.display.set_mode(
            (config.SURFACE_WIDTH, config.SURFACE_HEIGHT)
        )
```

```python
        # Load background image from file
        self.background = pygame.image.load("./content/street.png")

        # Set window caption
        pygame.display.set_caption("Car Crash")

        # Set window icon
        a = pygame.image.load('car.ico')
        pygame.display.set_icon(a)

        # Create the player and enemy sprites
        self.create_sprites()

        # Set up Fonts
        self.font_big = pygame.font.SysFont("Verdana", 60)
        self.font_small = pygame.font.SysFont("Verdana", 20)

        # Render this font now, it won't change and is only used once
        # Not rendering in the Game Loop increases performance
        self.game_over = self.font_big.render(
            "Game Over",
            True,
            CarCrash.BLACK
        )

        # Load sound file into memory
        pygame.mixer.music.load('content/background_music.wav')
        # Set volume to 30%, range from 0.0 (mute) to 1.0 (full volume)
        pygame.mixer.music.set_volume(0.3)
        # Play in a loop until stopped
        pygame.mixer.music.play(-1)
```

In the code above, we're setting up fonts to be used later in our program. We create two different fonts, **font_big** and **font_small** which both have the same font family, but different font sizes.

We use the **render()** function to actually create the graphics for the Font. We pass in the text we wish to be displayed and the color we want it to be in. In this case, **"Game Over"** and **BLACK** respectively.

```
98          # ------------------- DRAW ON BACKBUFFER -----------------------#
99          # Draw everything on the backbuffer first
100         # Fill the surface with the background image loaded earlier
101         self.surface.blit(self.background, (0, 0))
102
103         # Move and Re-draw all Sprites
104         for entity in self.all_sprites:
105             self.surface.blit(entity.image, entity.rect)
106             entity.move(self.dt)
107
108         # Render score before drawing it on the surface
109         scores = self.font_small.render(
110             str(self.enemy_sprite.score), True, CarCrash.BLACK)
111
112         # Draw score on the surface
113         self.surface.blit(scores, (10, 10))
114
115         # Move and Re-draw all Sprites
116         for entity in self.all_sprites:
117             self.surface.blit(entity.image, entity.rect)
118             entity.move(self.dt)
```

This is the second part of our fonts in the Game Loop. We render another font called **scores**. We didn't do this earlier because this font is meant to be rendered inside the Game Loop as it is continuously changing value. We moved the **game_over** font rendering out of the loop to avoid unnecessary performance loss.

We display both fonts using the **blit()** function. In it's first parameter we pass the rendered font and in the second we pass a pair of co-ordinates which mark the origin point of the font.

```python
        # ------------------- COLLISION ------------------------------------#
            # If a collision occurs between Player and Enemy
            if pygame.sprite.spritecollideany(
                self.player_sprite,
                self.enemies
            ):
                # Stop background sound
                pygame.mixer.music.stop()

                # Play crash sound
                pygame.mixer.Sound('content/crash.wav').play()

                # Wait 1 second
                sleep(1)

                # Fill the surface with RED
                self.surface.fill(CarCrash.RED)

                # Display game over on surface
                self.surface.blit(self.game_over, (30, 250))

                # Update the display
                pygame.display.update()

                # Kill all Sprites
                for entity in self.all_sprites:
                    entity.kill()

                # Wait 2 seconds
                sleep(2)

                # Exit the game
                pygame.quit()
                exit()

            # Redraw the surface
            pygame.display.update()
```

Sound

```python
# Background music plays continuously
pygame.mixer.music.load('background_music.wav')
pygame.mixer.music.set_volume(0.3)
pygame.mixer.music.play(-1)
```

The steps to playing background music.

1. Load the sound into memory.
2. Set the volume. The volume range is .0 - 1.
3. Play the sound in a continuous loop. The play argument `-1` loops the music.

```
# Stop background sound
pygame.mixer.music.stop()
```

4. Stop playing. We stop playing the background sound right before the crash sound that ends the game.

```
# Play crash sound
pygame.mixer.Sound('crash.wav').play()
```

We use a one-line function from the Pygame Mixer library to play a crash sound once a collision has occurred.

Enemy Class

```python
"""
    Name: enemy.py
    Author:
    Date:
    Purpose: All logic for the enemy's car is in this class
"""

import pygame
import random

# config.py contains global variables and constants
import config


class Enemy(pygame.sprite.Sprite):
    """Define the enemy class and methods"""
# ------------------------ INITIALIZE ENEMY OBJECT ------------------------#

    def __init__(self):
        self.score = 0

        # Construct an enemy object from Sprite class
        super().__init__()

        # Load enemy car image from file into a variable
        self.image = pygame.image.load("./content/enemy.png")

        # Create a surface rectangle a little smaller than the
        # enemy car image for better collision overlap 48x93
        self.surface = pygame.Surface((42, 83))

        # Create a rectangle with a random X location
        # Stay 40 pixels from the left and right edge
        self.rect = self.surface.get_rect(
            center=(random.randint(40, config.SURFACE_WIDTH - 40),  0)
        )
```

```python
    # -------------------------- MOVE-------------------------------------#
    def move(self, dt):
        # Move the sprite down SPEED pixels at a time
        self.rect.move_ip(0, config.speed * dt)

        # When the sprite reaches the botton of the surface
        if (self.rect.bottom > config.SURFACE_HEIGHT):
            # Move enemy back to the top of the screen
            self.rect.top = 0
            # Random center location on the X axis
            self.rect.center = (random.randint(30, 370), 0)

            # Increment the score every time the player dodges an oncoming car
            self.score += 1
```

We keep track of how many times the Enemy gets to the bottom of the screen. Each time this happens, the player gets one more point.

What's Next?

There is much more that can be done with this game. Here are some ideas for you to practice and implement on your own.

- Allow the player to restart the game.
- Keep track of the score between games.
- Multiple enemies spawning after set periods of time. (Similar to how we increased speed after a set period of time)
- Add some additional audio to the game, such as movement sounds (audio that plays when you move the character)
- Add the concept of multiple Lives or a Health bar.
- Variations in the shape and size of the "enemies".

Assignment Submission

Zip up the program files folder and submit in Blackboard.